Finding Similar Items in arXiv Abstracts

Algorithms for Massive Data – Project 1

Paolo Minini

Academic Year 2025/2026

Master in Data Science for Economics
Università degli Studi di Milano

Project Repository: `https://github.com/paolominini/lsh-abstract-similarity`

1 Introduction

The primary objective of this project is to build a scalable pipeline to detect highly similar scientific paper abstracts within the arXiv dataset. Finding similar documents in a corpus of big scale raises a computational problem: with over 3 million abstracts, a naive approach that computes the Jaccard similarity for all possible document pairs requires an $O(n^2)$ execution time. This method is completely infeasible. In addition to the time complexity, the implementation is strictly bound by hardware memory: the entire pipeline must execute successfully within the standard RAM limit of Google Colab, which is where the pipeline is executed. To bypass the bottleneck and keep memory usage strictly bounded, we implemented the following architecture:

1. **Shingling:** Parsing the normalized abstracts into sets of words to capture local semantic structure.
2. **MinHashing:** Compressing these variable-length sets into fixed-length numeric signatures using a family of hash functions.
3. **Locality-Sensitive Hashing (LSH):** Splitting the signatures into bands to hash similar documents into the same buckets, generating a highly reduced set of candidate pairs.
4. **Candidate Validation:** Computing the exact Jaccard similarity only on the isolated candidate pairs to filter out false positives.

2 Dataset & Pre-processing

The project utilizes the arXiv metadata snapshot (`Cornell-University/arxiv`), ingested directly via the Kaggle API. The full dataset contains over 3 million JSON objects.

2.1 Streaming Ingestion

To avoid memory crashes within the Google Colab RAM limits, data ingestion was built as a streaming process. The file is read line by line, parsing and filtering one abstract at a time. By processing data in this bounded way and immediately releasing unneeded memory, the RAM usage remains completely independent of the total corpus size.

2.2 Data Quality Filtering

During initial testing, we found that short arXiv withdrawal notices (e.g., "This paper has been withdrawn") caused a massive artificial spike in candidate pairs because their tiny shingle sets generated coarse, mathematically inflated Jaccard scores. To fix this, we integrated a filter directly into the streaming loop to drop any abstract with fewer than 20 words. This simple check removed 15,411 "invalid" abstracts, leaving a clean baseline of 3,064,847 valid abstracts.

2.3 Text Normalization

Before shingling, the raw text of each abstract must be normalized to create a clean, uniform vocabulary. We apply the following sequence to generate the final word tokens:

1. **Unicode Cleanup:** Text is decomposed using `NFKD` normalization. We apply an ASCII encode/decode step to drop orphaned combining marks, which prevents accented words (like "résumé") from being incorrectly split into separate tokens.

2. **Formatting Removal:** LaTeX math equations and structural commands are removed using regular expressions.

3. **Tokenization:** Non-alphanumeric characters and punctuations are stripped, the text is converted to lowercase, and extra whitespaces are collapsed to yield a final list of clean word tokens.

3 Shingling

To compute the Jaccard similarity between two abstracts, we represent each document as a set of k-shingles. A shingle is a contiguous sequence of tokens extracted from the normalized abstract using a sliding window.

3.1 Word-Level Shingling

We used 3-word shingles ($k = 3$) instead of character n-grams because words preserve the core technical concepts of the text. For instance, an abstract discussing "Natural Language Processing", "Large Language Models", or "Support Vector Machines" will produce shingles that map directly to these specific multi-word scientific entities. Character n-grams would instead chop these phrases into arbitrary letter sequences.

The tradeoff is that usual academic phrases (such as "state of the art" or "in this paper we") appear identically across completely unrelated documents. Because an abstract contains a relatively small total number of words, sharing even a few of these common phrases creates a small but artificial mathematical overlap. This property likely inflates our initial baseline false-positive rate. We explicitly accepted this tradeoff because word shingles align directly with real scientific concepts, making our final document matches highly interpretable.

3.2 Shingles Hashing

Storing sets of strings for a large amount of documents would quickly exceed the memory limits. To keep the memory footprint small and bounded, we hash each string shingle into a single 4-byte integer. We implemented this using Python's standard library `zlib.crc32` function. The unique shingles are stored in a set to remove duplicates, and then efficiently loaded into a NumPy `uint32` array using `np.fromiter`. By converting the text into lightweight 32-bit integers, each document becomes a small, predictable numeric array.

4 Vectorized MinHash Implementation

Once the abstracts are represented as arrays of 32-bit hashed word shingles, we compress these sets into fixed-length numeric signatures using MinHashing.

4.1 Theoretical Foundation

The core property of MinHashing is that the probability of collision between two hashed sets under a random permutation function h is exactly equal to their Jaccard similarity:

$$P(h(A) = h(B)) = J(A, B) \tag{1}$$

where $h(A)$ and $h(B))$ denote the minimum hash value computed over set A and B respectively. By computing M independent random permutations, we can estimate the true Jaccard similarity of any document pair simply by counting the fraction of matching entries in their signature vectors. For this project, we selected $M = 100$ hash functions, compressing each abstract into a signature vector of length 100.

Rather than explicitly shuffling the massive shingle vocabulary, a random permutation is simulated using a linear hash function family:

$$h_{a,b}(x) = (a \cdot x + b) \pmod{p} \tag{2}$$

where x is the 32-bit shingle integer, $p = 2^{32} - 5$ is the largest prime number strictly smaller than 2^{32} to ensure a uniform distribution and minimize collisions, and a, b are randomly generated integers.

4.2 Engineering application

A naive implementation of MinHashing requires a nested loop structure over every hash function, every document, and every shingle, which would run incredibly slowly in pure Python. To build a scalable implementation, we fully vectorized the algorithm and combined it with an on-disk memory map to prevent RAM overflow.

First, the entire $N \times 100$ signature matrix is initialized directly on the hard drive using `np.memmap`. The abstracts are then processed in bounded batches. For a given batch, all document shingles are concatenated into a single flat array. We compute the linear hash transformations by broadcasting the random coefficient vectors a and b across this entire flat array simultaneously.

To find the minimum hash value for each signature slot per document without looping, we use NumPy's highly optimized reduction function `np.minimum.reduceat`. By passing the index boundaries that indicate where each document begins and ends in the flat array, NumPy computes the row-wise minimums in a single step. The computed signatures for the batch are then written straight to the disk map, and the working memory is cleared for the next batch.

5 LSH Banding

After generating the signature matrix, finding similar documents still presents an $O(n^2)$ pairing problem if we compare the signatures directly. Locality-Sensitive Hashing (LSH) solves this by mapping similar signatures into identical hash buckets, isolating a small set of candidate pairs.

5.1 Theoretical Foundation

The signature matrix of size $N \times M$ (where $N = 3,064,847$ documents and $M = 100$ hash functions) is partitioned into b horizontal bands, each containing r rows, such that $M = b \cdot r$.

Documents that share the exact same r values in at least one band b are flagged as candidates. For two documents with a true Jaccard similarity s, the probability that their signature fragments match perfectly in at least one band is defined by the following formula:

$$P(\text{candidate}) = 1 - (1 - s^r)^b \tag{3}$$

The threshold of this curve defines the theoretical similarity boundary t, which marks the steepest point of the curve (the similarity at which a pair's candidate probability rises sharply):

$$t = \left(\frac{1}{b}\right)^{\frac{1}{r}} \tag{4}$$

Our primary objective is to capture pairs with a high similarity threshold while managing the false-positive rate. We configured the banding parameters to $b = 20$ bands and $r = 5$ rows. This choice yields a theoretical threshold of:

$$t = \left(\frac{1}{20}\right)^{\frac{1}{5}} \approx 0.549 \tag{5}$$

This mathematical configuration matches our empirical target, ensuring that pairs with a similarity around 0.5 or higher are flagged as candidates with high probability. This characteristic behavior is visualized by the S-curve in Figure 1.

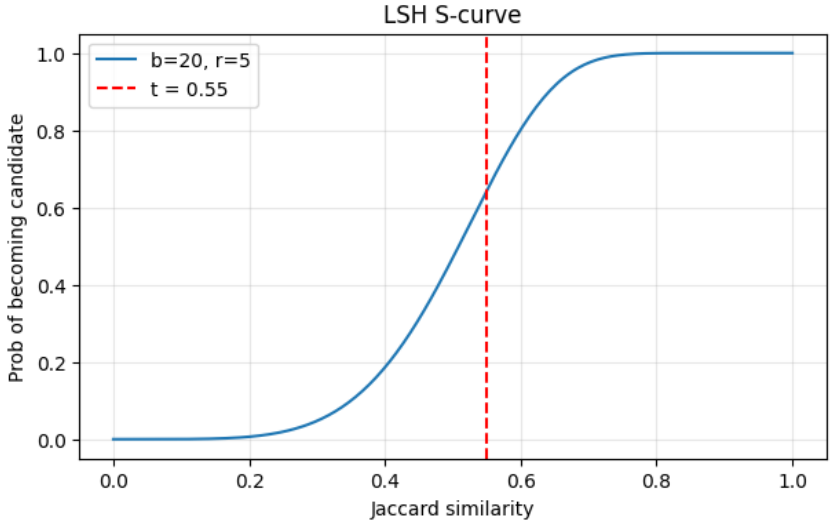

Figure 1: Theoretical LSH S-curve showing the probability of becoming a candidate pair as a function of exact Jaccard similarity, configured for $b = 20$ bands and $r = 5$ rows. The threshold is marked at $t \approx 0.549$.

5.2 Engineering application

With the full signature matrix safely stored on disk via `np.memmap`, the LSH phase processes the data band by band to avoid loading the massive matrix into active RAM.

For each of the 20 bands, we read a contiguous slice of 5 columns (representing the $r = 5$ rows for that band) across all 3 million documents. We then iterate through the documents, converting their 5-integer signature slice into a raw byte string using NumPy's `tobytes()` method. This byte string serves as a deterministic bucket ID.

We use a Python dictionary of lists to group the indices of documents that share the exact same byte string. Finally, for any bucket containing more than one document, we apply

`itertools.combinations` to instantly extract all unique document pairings. By handling only a single 5-column band in memory at any given time, the system footprint stays flat, completing the millions of necessary comparisons without triggering Colab hardware limits.

6 Validation & Experimental Results

After the LSH stage generated the set of candidate pairs, the final step in the pipeline is to filter out the false positives and identify the true near-duplicates.

6.1 Precision Methodology

The LSH banding process successfully reduced the 3,064,847 documents down to a highly concentrated list of 19,875 candidate pairs. To validate these matches and measure the precision of our pipeline, we extracted a reproducible random sample of 1,000 candidate pairs.

For this sample, we computed the exact Jaccard similarity directly on the original 3-word shingle sets of the flagged documents. Any candidate pair with an exact Jaccard similarity below our theoretical threshold of 0.549 is classified as a false positive. Evaluating this sample yielded a final precision of 62.6%.

6.2 Candidate Quality

To evaluate the health of the pipeline, we plotted a histogram of the exact Jaccard similarities for the sampled candidate pairs, as shown in Figure 2. Because we intentionally used word-level shingles, our raw candidate pool naturally contained false positives driven by usual academic phrases. This explains the remaining pairs that fell below the threshold, which are easily stripped away during this final exact-Jaccard check.

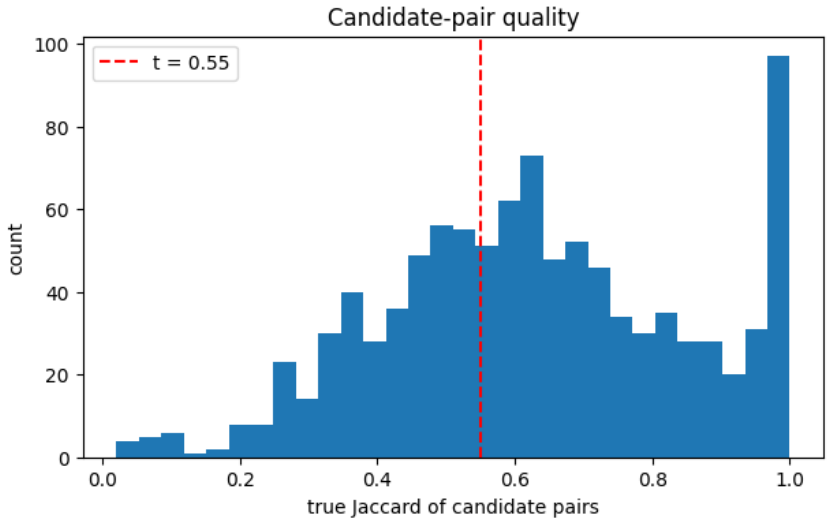

Figure 2: Distribution of exact Jaccard similarities for a random sample of 1,000 candidate pairs.

7 Scalability & Limitations

To prove the pipeline works within Google Colab's 12.7 GB RAM limit, we benchmarked execution time and memory (Resident Set Size) on increasing dataset sizes. The results are summarized in Table 1 and Figure 3.

Documents (n)	MinHash Time	LSH Time	Total Time	Candidates
10,000	5.8 s	0.2 s	6.0 s	30
20,000	7.9 s	0.4 s	8.3 s	70
50,000	24.9 s	2.5 s	27.4 s	296
100,000	44.1 s	5.6 s	49.8 s	758
3,064,847	28.4 min	2.6 min	31.0 min	19,875

Table 1: Benchmark results showing linear time scaling and flat memory usage.

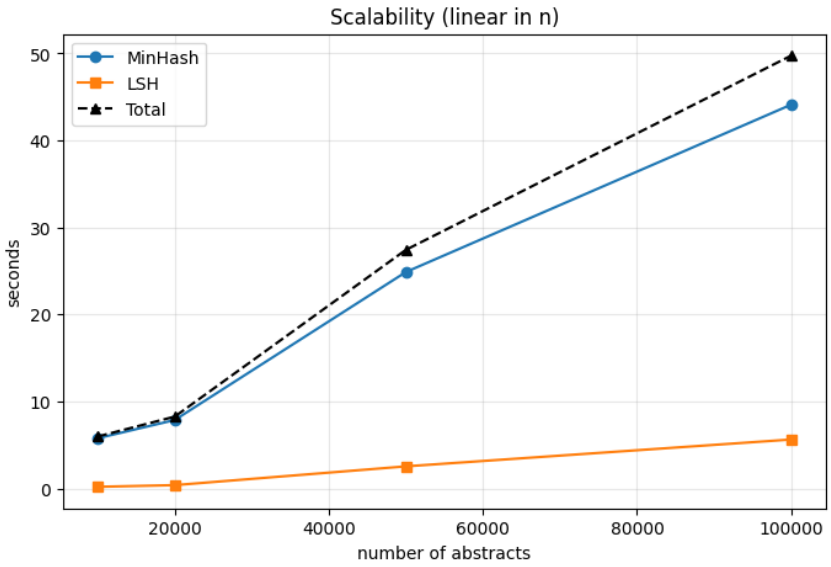

Figure 3: Execution time of the MinHash and LSH stages relative to dataset size.

7.1 Time and Memory Performance

As the data shows, we successfully bypassed the $O(n^2)$ bottleneck. Both the MinHash and LSH stages scale near-linearly. Figure 3 plots sizes up to 100,000 documents to make the near-linear trend visually legible, while Table 1 includes the full 3,064,847 document baseline, which completed in roughly 30 minutes.

In additon, active RAM usage stayed perfectly flat at around 4.5 GB. Because we computed the signatures in small chunks and saved them directly to disk, the LSH algorithm itself did not accumulate any extra RAM, keeping the project completely safe from crashes.

7.2 Limitations

The main tradeoff of our approach is using word-shingles. Common academic formulations force inflated similarities between entirely unrelated papers.

8 Conclusion

This project successfully implemented a scalable near-duplicate detection pipeline for over 3 million arXiv abstracts. By combining streaming data ingestion, vectorized MinHash signatures, and memory-mapped LSH banding, we bypassed the infeasible $O(n^2)$ comparison bottleneck and operated safely within strict hardware limits. Despite the false-positive tradeoff inherent to word-level shingles, the architecture efficiently isolated true duplicate candidates, ultimately achieving a final validation precision of 62.6% without ever exceeding 4.5 GB of active RAM.

I/We declare that this material, which I/We now submit for assessment, is entirely my/our own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my/our work, and including any code produced using generative AI systems. I/We understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study.